How can we measure web applications and development workflows?

Lecture 3 · Establish

A number becomes evidence only after its question and boundary are clear.

Start with the decision the measurement supports

Define:

- the user or engineering question;
- the system boundary;
- the workload and environment;
- the observable signal;
- the comparison or threshold;
- what the result cannot establish.

Collecting available metrics first often produces precise answers to the wrong question.

Runtime evidence needs shared vocabulary

Concept	Meaning
Load	Work offered to the system
Throughput	Work completed per unit of time
Latency	Time one unit of work spends in the system
Errors	Work that fails or produces an unacceptable result
Saturation	A constrained resource approaches or reaches its limit
Resource use	CPU, memory, storage, network, energy, or spend consumed

Distributions reveal what averages hide

A mean latency can improve while the slowest accepted requests become worse.

Report:

- the full set of repetitions;
- median or another declared center;
- tail values such as p95 or p99 when relevant;
- fastest-to-slowest range;
- errors and discarded runs;
- the exact revision and environment.

Logs, metrics, and traces answer different questions

Logs preserve discrete events and explanations.

Metrics summarize behavior over time.

Traces connect work across boundaries and dependencies.

Observability is the ability to investigate relevant internal state from available signals—not the number of dashboards installed.

Little’s Law checks whether averages agree

$L = \lambda W$

If 20 events arrive each second and each spends 0.5 seconds in the system:

L = 20 × 0.5 = 10 events in progress on average

Use the relationship only over a stable observation window with a clearly defined boundary.

It checks consistency; it does not identify the cause of delay.

A repeatable protocol makes later comparison possible

Keep constant where practical:

- acceptance behavior and workload;
- warm-up and repetition rules;
- dataset shape and size;
- environment and tool versions;
- collected signals and reporting format.

Record differences rather than hiding them when perfect control is impossible.

Complete Establish with a baseline

Submit:

- a reproducible implementation A;
- a repeatable runtime workload;
- development-workflow evidence;
- operational run evidence;
- the two bounded claims and their current support;
- an explicit statement of what the baseline does not prove.

Question to carry forward: Which architectural assumption created the observed behavior?